VIVA & PROJECT DEFENSE QUESTION BANK

12 Dataset & Model Questions — Easy Answers

Project: AI-Based Cybersecurity Threat Detection • Dataset: NSL-KDD (148,517 Network Connections)

Q1. What dataset are you using for your project? Where did you get it from?

Easy Answer (In Simple Words):
We are using the NSL-KDD Dataset (Network Security Laboratory - Knowledge Discovery in Databases), the gold-standard benchmark for network intrusion detection.

- **Where We Got It From:**
  Created and published by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB) as an improved, cleaned version of the original DARPA / KDD Cup '99 dataset.
- **How Our Project Downloads It:**
  Our ingestion script (src/data_ingestion.py) automatically downloads KDDTrain+.txt and KDDTest+.txt from the public NSL-KDD benchmark repository and stores them in data/raw/.
- **What Each Row Represents:**
  Each row represents one network connection (TCP, UDP, or ICMP flow) labeled as legitimate ('normal') or one of 39 cyber-attack subtypes grouped into 4 major attack families.

`Project File: src/data_ingestion.py | Files: data/raw/KDDTrain+.txt & data/raw/KDDTest+.txt`

Q2. What is the size of your dataset? How many rows / images / samples?

Easy Answer (In Simple Words):
Our dataset is a structured tabular dataset (not images) containing a total of 148,517 network connection samples (rows) and 43 raw columns (~22.5 MB raw text).

- **Training Set Size (KDDTrain+.txt):**
  125,973 rows (84.8% of total data) — consists of 67,343 Normal connections and 58,630 Attack connections.
- **Testing Set Size (KDDTest+.txt):**
  22,544 rows (15.2% of total data) — consists of 9,711 Normal connections and 12,833 Attack connections (including 17 brand-new zero-day attack types not present in training).
- **Columns / Dimensions:**
  Each raw sample has 41 network traffic features + 1 attack label + 1 difficulty score = 43 raw columns. After feature engineering, each sample has 48 normalized numeric features.

`Total Samples: 148,517 rows | Processed Matrix Shapes: X_train (125973, 48) & X_test (22544, 48)`

Q3. What are the features / attributes in your dataset?

Easy Answer (In Simple Words):
Each connection has 41 raw network features (3 categorical text + 38 numeric) plus 7 new behavioral features we engineered, making 48 total input features.

- **1. Basic Connection Features (Features 1-9):**
  Packet header details: duration, protocol_type (tcp/udp/icmp), service (http, ssh, dns, ftp...), flag (SF, S0, REJ), src_bytes (bytes sent), dst_bytes (bytes received), land, wrong_fragment, urgent.
- **2. Content & Host Intrusion Clues (Features 10-22):**
  Inside-payload clues: hot indicators, num_failed_logins, logged_in, num_compromised, root_shell (root access gained!), su_attempted, num_root, num_file_creations, is_guest_login.
- **3. Traffic Window Features (23-41) + 7 Engineered Features:**
  2-second window stats (count, srv_count, serror_rate, rerror_rate, same_srv_rate) + 255-connection host stats + our 7 derived features (bytes_ratio, traffic_intensity, srv_diversity, total_error_rate, suspicious_activity).

`Target Labels: Binary (0=Normal, 1=Attack) & 5-Class (Normal, DoS, Probe, R2L, U2R)`

Questions 4, 5 & 6: Data Source, Class Imbalance & Preprocessing

Q4. Is your dataset public or did you collect it yourself?

Easy Answer (In Simple Words):
NSL-KDD is a publicly available, peer-reviewed academic benchmark dataset; additionally, our project generates live custom packet telemetry in our Threat Sandbox.

- **Why We Used a Public Benchmark Instead of Self-Collection:**
  Collecting 148,000+ real cyber attacks (like Rootkits, SYN Floods, and Buffer Overflows) on a private network takes months and risks damaging live infrastructure.
- **Scientific Reproducibility:**
  Using the public NSL-KDD benchmark allows us to compare our model's accuracy (80.74% DNN, 0.9723 AUC-ROC) fairly against thousands of published IEEE and ACM research papers.
- **Self-Generated Live Sandbox Data:**
  On top of the public dataset, we built an interactive Threat Sandbox (/api/predict) and Real-Time Traffic Simulator (src/realtime_simulator.py) so users can inject their own custom packets!

`Source Type: Public Benchmark (NSL-KDD) + Custom Live Packet Injection via REST API`

Q5. Is your dataset balanced or imbalanced? How did you handle it?

Easy Answer (In Simple Words):
For Binary classification it is well-balanced (53.5% Normal vs 46.5% Attack), but for 5-Class classification it is heavily imbalanced (U2R is only 0.04% of rows).

- **Exact Class Distribution in Training Set (125,973 rows):**
  Normal: 67,343 (53.46%) | DoS: 45,927 (36.46%) | Probe: 11,656 (9.25%) | R2L: 995 (0.79%) | U2R: 52 (0.04%). Notice how rare R2L and U2R attacks are!
- **How We Handled It — 1. Balanced Class Weighting:**
  In src/model_training.py, we set class_weight='balanced' inside RandomForestClassifier. This penalizes mistakes on rare classes (U2R/R2L) up to 1,000x more heavily than majority classes.
- **How We Handled It — 2. Unsupervised Isolation Forest + Composite Features:**
  We trained an Isolation Forest ONLY on Normal traffic so rare attacks are caught as statistical outliers regardless of sample size, and created 'suspicious_activity' to amplify R2L/U2R signals.

`Evaluation Strategy: We use Weighted F1-Score, Precision, Recall & FPR instead of raw accuracy alone.`

Q6. What preprocessing did you do? (cleaning, null values, normalization)

Easy Answer (In Simple Words):
We performed 4 preprocessing stages in src/feature_engineering.py: cleaning/null-handling, categorical encoding, creating 7 derived features, and MinMax normalization.

- **1. Cleaning & Preventing Null / Division-by-Zero Errors:**
  Verified zero missing values in raw files and added Laplace smoothing (+1 in denominators like src_bytes / (dst_bytes + 1)) so zero-byte packets never produce NaN or infinity errors.
- **2. Categorical Label Encoding (Text -> Numbers):**
  Used Scikit-Learn LabelEncoder to convert the 3 text columns — protocol_type ('tcp','udp','icmp'), service (70 services), and flag (11 TCP states) — into numeric columns.
- **3. Feature Engineering (+7 Clues) & MinMax Normalization [0.0 to 1.0]:**
  Engineered 7 ratios (bytes_ratio, traffic_intensity, srv_diversity, total_error_rate, etc.) and applied MinMaxScaler() fitted ONLY on training data so large byte counts don't dominate small rates.

`Saved Transformers: models/saved/scaler.pkl, encoders.pkl & category_encoder.pkl`

Questions 7, 8 & 9: Train/Test Split, Ethics/Privacy & Dataset Choice

Q7. How did you split your dataset for training and testing?

Easy Answer (In Simple Words):
We used the official benchmark split (~85% Training = 125,973 rows, ~15% Testing = 22,544 rows), where the test set includes 17 unseen zero-day attack types.

- **Why We Used the Official Split Instead of a Random 80/20 Split:**
  A random train_test_split puts the exact same attack types in both train and test, giving an unrealistic 99.5% score. The official KDDTest+ split holds out 17 novel attack types ONLY in the test set!
- **Validation Split During Deep Neural Network Training:**
  Inside the 125,973 training records, our Deep Neural Network reserves an additional 15% validation split (validation_split=0.15) for EarlyStopping and learning-rate tuning.
- **Strict Zero Data-Leakage Policy:**
  Our MinMaxScaler and LabelEncoders are fit() strictly on KDDTrain+ and only transform() is called on KDDTest+, ensuring zero test information leaks into training.

`Split Breakdown: 125,973 Train (with 15% internal val split for DNN)  vs  22,544 Unseen Test`

Q8. Are there any ethical / privacy issues with your dataset?

Easy Answer (In Simple Words):
The dataset is completely anonymized (no real personal IPs, names, or payload text), and we added strict ethical guardrails for automated IP blocking.

- **1. Complete User Privacy (No PII or Packet Payloads):**
  NSL-KDD contains only statistical flow metadata (byte counts, durations, flags). It does NOT contain private user emails, passwords, web URLs, or personally identifiable IP addresses.
- **2. Ethical Risk of False Positives in Automated Blocking:**
  In real-world cybersecurity, if an AI falsely flags a normal user (False Positive) and auto-blocks their IP, it can disrupt hospital, banking, or emergency traffic.
- **3. How Our System Mitigates Ethical & Security Risks:**
  We only auto-quarantine 'Critical' high-confidence threats, provide a 1-click Unblock button in the dashboard, and enforce read-only SQL guardrails (src/siem_db.py) to prevent data tampering.

`Privacy Status: 100% Anonymized Flow Metadata (GDPR-safe) + Human-in-the-Loop Unblock Control`

Q9. Why did you choose this particular dataset and not others?

Easy Answer (In Simple Words):
We chose NSL-KDD because it fixes the duplicate-record flaws of KDD'99, covers all 4 core attack families, tests zero-day generalization, and trains fast on standard hardware.

- **1. Superior to Original KDD Cup '99:**
  The older KDD'99 dataset had 78% duplicate records, which caused models to memorize frequent rows. NSL-KDD removed all duplicates so models learn genuine network behavior.
- **2. Practical Size Compared to Multi-Gigabyte Captures (CICIDS2017 / UNSW-NB15):**
  While datasets like CICIDS2017 are 10+ GB and require heavy GPU clusters, NSL-KDD (~148k rows) lets us train 5 diverse models in under 60 seconds and run real-time sub-millisecond inference.
- **3. Built-In Zero-Day Stress Test:**
  Because KDDTest+ contains 17 attack subtypes missing from KDDTrain+, it is the best dataset to prove why combining Supervised Models + Unsupervised Isolation Forest is necessary.

`Key Advantage: Clean, zero-duplicate benchmark with built-in zero-day attack evaluation.`

Questions 10, 11 & 12: Limitations, Future Data & Real-Time Deployment

Q10. What are the limitations of your dataset?

Easy Answer (In Simple Words):
Its three main limitations are its age (older network protocols), extreme scarcity of U2R samples (only 52 training rows), and lack of modern encrypted TLS/cloud attack traffic.

- **1. Age of Traffic & Modern Encrypted Protocols:**
  NSL-KDD reflects classic TCP/IP attacks (like Telnet, FTP, and Smurf DoS) and lacks modern cloud-native attacks, API exploits, or encrypted HTTPS/QUIC malware traffic.
- **2. Severe Scarcity of U2R & R2L Samples:**
  With only 52 U2R (User-to-Root) samples out of 125,973 training rows, supervised models struggle to learn every possible privilege-escalation variation without heuristic help.

`How We Compensated: Added Isolation Forest anomaly scoring + heuristic flag/root indicators.`

Q11. If you get more data in future, how will it improve your model?

Easy Answer (In Simple Words):
More data—especially rare U2R/R2L attacks and modern cloud/IoT traffic—will dramatically boost multi-class Recall, cut False Positives, and stop concept drift.

- **1. Higher Recall on Rare Attacks (R2L & U2R):**
  Adding thousands of modern SSH brute-force (R2L) and container privilege-escalation (U2R) examples will raise multi-class accuracy from 76.74% toward 95%+.
- **2. Continuous Learning & Reduced False Alarms:**
  Retraining our pipeline (python3 run_pipeline.py --train) on fresh company network baselines will teach the Isolation Forest new normal user habits, lowering False Positive Rates below 0.5%.

`Retraining Command: ./venv/bin/python run_pipeline.py --train  (Rebuilds all 5 models automatically)`

Q12. Can your model work on real-time / new data, not just this dataset?

Easy Answer (In Simple Words):
YES! Our model is already deployed behind a live Flask REST API (/api/predict) and SSE stream that classifies brand-new, custom network connections in under 15 milliseconds.

- **1. Raw Connection Inference Pipeline (classify_raw_connection):**
  In src/threat_classifier.py, our classify_raw_connection() method accepts any new JSON packet dictionary, fills missing defaults, computes the 7 derived features, encodes, scales, and predicts live!
- **2. Live Threat Sandbox, SIEM Database & Firewall Integration:**
  Users can inject custom traffic via the Web Dashboard or curl POST /api/predict. Critical attacks are logged to SQLite (siem_telemetry.db) and exported to iptables, UFW, and pf.conf firewall rules.

```
Live Test: curl -X POST localhost:5050/api/predict -d '{"protocol_type":"tcp","flag":"S0","count":250}'
```

FAST NUMBERS CHEAT-SHEET FOR YOUR VIVA / PRESENTATION

• **Dataset Name:**	NSL-KDD (Canadian Inst. for Cybersecurity)	• **Total Samples:**	148,517 rows (125,973 Train + 22,544 Test)
• **Feature Count:**	48 Features (41 Raw NSL-KDD + 7 Derived)	• **Target Classes:**	5 Classes (Normal, DoS, Probe, R2L, U2R)
• **5 Models Used:**	RF (Bin/Multi), DNN (Bin/Multi), IsoForest	• **Top Metrics:**	80.74% Acc (DNN) | 98.53% Prec (IsoForest)
• **Imbalance Fix:**	class_weight='balanced' + Unsupervised Isolation Forest + Composite Behavioral Features		